

Interview Preparation Guide

LangChain • LangGraph • LangSmith • Vector Databases • LoRA & QLoRA • Practical & HR Questions

Roman Urdu mein DETAILED Questions & Answers - Lahore based company, 1-year experience role ke liye - Kal ke interview ki mukammal tayyari.

Is guide mein shamil topics:

- 1. LangChain (18 Questions)
- 2. LangGraph (12 Questions)
- 3. LangSmith (10 Questions)
- 4. Vector Databases (14 Questions)
- 5. LoRA (Low-Rank Adaptation) (11 Questions)
- 6. QLoRA (Quantized LoRA) (8 Questions)
- 7. Practical / Scenario-Based Questions (10 Questions)
- 8. Foundational AI/ML Concepts (10 Questions)
- 9. HR / General Interview Questions (10 Questions)

1. LangChain

LangChain ek open-source framework hai jo LLM (Large Language Model) based applications banane ke liye use hota hai. Ye LLMs ko external data, tools, APIs aur memory ke sath connect karne ka asaan tareeqa deta hai, taake sirf chatbot nahi balke real-world agents aur RAG systems bhi bana sakein.

Q1. LangChain kya hai aur ye kis problem ko solve karta hai?

LangChain ek framework hai jo developers ko LLM-powered applications jaldi build karne mein madad karta hai. Asal mein LLMs sirf text input/output tak limited hote hain - unko apna koi memory, external data ya tools access nahi hota. LangChain ye gap fill karta hai: ye LLM ko documents, databases, APIs aur tools ke sath chain karke real applications (jaise chatbots, RAG systems, agents) banane deta hai.

Q2. LangChain ke main components/modules kya hain?

LangChain ke core building blocks ye hain: Models (LLM ya chat model wrappers), Prompts (prompt templates), Chains (multiple steps ko sequence mein jorna), Memory (conversation history store karna), Indexes/Retrievers (documents ko search karna), Agents (LLM khud decide kare konsa tool use karna hai), aur Callbacks (logging, monitoring ke liye).

Q3. Chain kya hoti hai LangChain mein, example ke sath samjhao?

Chain ek sequence hoti hai jismein multiple components ek dusre ke sath linked hote hain - jaise prompt template -> LLM call -> output parser. Sab se simple example LLMChain hai jo ek prompt leta hai, LLM ko bhejta hai, aur response wapis deta hai. SequentialChain mein multiple chains ek ke baad ek chalti hain, jahan ek chain ka output dusri chain ka input banta hai.

Q4. Agent aur Chain mein kya farak hai?

Chain mein steps fixed/predetermined hote hain - har baar same order mein chalte hain. Agent mein LLM khud reasoning kar ke decide karta hai ke kaunsa tool kab use karna hai aur kis order mein (ReAct pattern: Reason + Act). Matlab agent dynamic decision making karta hai, chain static execution path follow karti hai.

Q5. Memory ka role kya hai LangChain mein, aur uski types batao?

Memory conversation ki history store karne ke liye use hoti hai taake chatbot pichli baatein yaad rakh sake. Common types: ConversationBufferMemory (puri history raw save karti hai), ConversationSummaryMemory (history ko summarize karke rakhti hai taake tokens kam lagein), aur ConversationBufferWindowMemory (sirf last N messages yaad rakhti hai).

Q6. RAG (Retrieval Augmented Generation) LangChain mein kaise kaam karta hai?

RAG mein pehle documents ko chunks mein toda jata hai, phir har chunk ko embedding (vector) mein convert karke vector database mein store karte hain. Jab user query karta hai, query ko bhi embed kar ke vector store se relevant chunks retrieve kiye jate hain. Ye retrieved context LLM ko prompt ke sath diya jata hai taake LLM accurate aur grounded answer de sake, sirf apni training knowledge pe depend na kare.

Q7. LangChain mein Retriever kya hota hai?

Retriever ek component hai jo query ke basis pe vector store (ya kisi aur source) se relevant documents fetch karta hai. Ye similarity search use karta hai - query ka embedding bana kar database mein closest matching vectors dhundta hai.

Q8. LangChain mein Tools aur Toolkits kya hote hain?

Tools wo external functions ya APIs hote hain jo agent use kar sakta hai - jaise web search, calculator, database query, ya koi custom Python function. Toolkit multiple related tools ka collection hota hai, jaise SQL toolkit jismein query run karna, schema dekhna, etc شامل hote hain.

Q9. Output Parser ka kya use hai?

LLM ka output normally raw text hota hai. Output parser is text ko structured format mein convert karta hai - jaise JSON, list, ya kisi custom Pydantic model mein - taake application code usay easily use kar sake.

Q10. LangChain Expression Language (LCEL) kya hai?

LCEL LangChain ka declarative way hai chains banane ka, jahan components ko pipe operator (|) se connect karte hain, jaise prompt | model | output_parser. Ye streaming, async, aur parallel execution ko asaan banata hai, aur production-grade chains ke liye recommended approach hai.

Q11. ConversationalRetrievalChain kya hoti hai?

Ye chain RAG aur chat memory dono ko combine karti hai. Jab user follow-up question puchta hai (jisme pichli baat ka reference ho, jaise 'aur uski price kya hai?'), ye pehle question ko chat history ke sath standalone question mein reformulate karti hai, phir us reformulated question se vector store se relevant documents retrieve karti hai, aur phir LLM se answer generate karti hai.

Q12. LangChain mein different LLM providers (OpenAI, Anthropic, HuggingFace) ko kaise integrate karte hain?

LangChain har provider ke liye standard wrapper classes deta hai (jaise ChatOpenAI, ChatAnthropic, HuggingFaceEndpoint) jo sab same common interface follow karte hain (invoke, stream, batch methods). Isliye agar kal ko provider switch karna ho (jaise OpenAI se Anthropic), to bohot kam code change hota hai.

Q13. LangChain mein streaming response kaise implement karte hain?

Model object pe .stream() method call karte hain jo tokens ko real-time generate hote hi yield karta hai, ya StreamingStdOutCallbackHandler jaisi callbacks attach karte hain. Isse user ko poora response ek sath wait karne ki bajaye, word-by-word dikh jata hai - jaise ChatGPT mein hota hai.

Q14. Document Loaders kya hain LangChain mein?

Document Loaders alag alag sources (PDF, CSV, Word files, websites, Notion, databases, etc.) se raw data load karke unhe standard 'Document' object mein convert karte hain, taake aage chunking aur embedding ke liye consistent format mil sake.

Q15. Text Splitters kya hote hain aur kyun zaroori hain?

Text Splitters bare documents ko chote chunks mein todte hain, kyunke ek to LLM ka context window limited hota hai, dusra chote chunks se retrieval zyada precise hoti hai. RecursiveCharacterTextSplitter sab se commonly use hota hai kyunke ye pehle paragraph, phir sentence, phir word boundaries pe split karne ki koshish karta hai taake meaning na tute.

Q16. LangChain mein Callback Handlers ka kya use hai?

Callbacks intermediate events (jaise LLM start/end, tool call, error) ko hook karne deti hain - inse custom logging, token usage tracking, ya real-time UI updates implement kar sakte hain bina core chain logic ko touch kiye.

Q17. LangChain mein agent ka ReAct pattern kya hota hai?

ReAct ka matlab hai Reasoning + Acting. Agent ek loop follow karta hai: pehle 'Thought' (sochta hai ke kya karna hai), phir 'Action' (koi tool call karta hai), phir 'Observation' (result dekhta hai), aur ye cycle tab tak chalta hai jab tak final answer nahi mil jata.

Q18. LangChain mein Pydantic ya schema validation kyun use ki jati hai?

Jab LLM se structured output chahiye (jaise JSON), to Pydantic models define karke 'with_structured_output' ya output parser ke sath use karte hain, taake LLM ka response automatically validate aur correct data types mein parse ho jaye, manual string parsing ki zaroorat na pare.

2. LangGraph

LangGraph LangChain team ki banayi hui library hai jo complex, stateful aur multi-step LLM workflows ko graph ki form mein design karne deti hai. Ye normal linear chains ki limitation door karti hai aur loops, branching aur multi-agent systems support karti hai.

Q1. LangGraph kya hai aur ye kyun banaya gaya?

LangGraph ek library hai jo LLM applications ko graph structure mein build karne deti hai, jahan nodes (steps/functions) hote hain aur edges decide karte hain ke control flow kaise chalega. Ye isliye banaya gaya kyunke traditional LangChain agents simple linear ya loop-based reasoning tak limited the - complex, multi-step aur multi-agent workflows ke liye zyada control aur flexibility chahiye thi.

Q2. LangGraph traditional LangChain Agents se kaise different hai?

Traditional agents ek fixed ReAct-type loop follow karte hain (think -> act -> observe), jahan developer ko zyada control nahi milta. LangGraph mein developer khud graph design karta hai - explicitly define karta hai konsa node kab chalega, konsa condition pe konsa path lena hai. Isse cycles, parallel branches, aur human-in-the-loop steps add karna bohot asaan ho jata hai.

Q3. LangGraph mein 'State' kya hota hai?

State ek shared data structure (typically a TypedDict ya Pydantic model) hoti hai jo poore graph ke through pass hoti hai. Har node state ko read karta hai, kuch process karta hai, aur state ko update kar deta hai. Ye state hi workflow ki memory ka kaam karti hai.

Q4. Node aur Edge ka kya matlab hai LangGraph mein?

Node ek function ya step hota hai jo state lekar koi kaam karta hai (jaise LLM call ya tool call) aur updated state return karta hai. Edge ye define karta hai ke ek node ke baad konsa node chalega. Edges normal (fixed) bhi ho sakti hain aur conditional bhi.

Q5. Conditional Edges kya hote hain aur unka use kya hai?

Conditional edges dynamic routing allow karte hain - matlab state ya output ke basis pe decide hota hai ke next kaunsa node chalega. Ye if/else jaisi branching logic graph ke andar implement karne deta hai, jaise agar tool call zaroori ho to tool node pe jao, warna directly end node pe jao.

Q6. LangGraph mein Checkpointing/Persistence kya hai?

Checkpointing ka matlab hai state ko har step pe save karna, taake workflow ko baad mein wahi se resume kiya ja sake jahan rok gaya tha. Ye long-running conversations, error recovery, aur human-in-the-loop scenarios (jahan insaan beech mein approve/reject kare) ke liye bohot useful hai.

Q7. LangGraph mein Multi-Agent systems kaise design karte hain?

Har agent ko ek separate node ki tarah define kiya jata hai, jisme apna specific role hota hai (jaise researcher agent, writer agent, reviewer agent). Phir edges se decide karte hain ke ek agent ka output kis agent ko handoff hoga. Supervisor pattern bhi common hai jahan ek 'supervisor' node decide karta hai konsa agent next chalega.

Q8. Cycles allow karne ka kya faida hota hai LangGraph mein?

Cycles se iterative refinement possible hota hai - jaise agent apna output generate kare, phir khud check kare ya kisi 'critic' node se review karaye, aur agar sahi na ho to dobara try kare. Ye retry logic, self-correction, aur quality improvement ke liye essential hai jo simple linear chain mein possible nahi tha.

Q9. LangGraph mein Human-in-the-loop kaise implement karte hain?

Graph mein ek specific point pe 'interrupt' laga dete hain jahan execution ruk jata hai aur insaan ko output review/edit/approve karne ka mauqa milta hai. Insaan ke response ke baad (checkpoint se) graph wahin se resume ho jata hai. Ye sensitive actions (jaise payment, email send karna) ke liye bohot important pattern hai.

Q10. START aur END nodes kya hote hain LangGraph mein?

Ye special predefined nodes hain jo graph ka entry point (START) aur exit point (END) define karte hain. Har graph ko build karte waqt batana parta hai ke execution START se kahan begin hogi aur kaunse node ke baad END pe pohanch kar ruk jayegi.

Q11. LangGraph mein Reducer functions ka kya kaam hai?

Jab multiple nodes ek hi state key ko update karte hain, reducer function decide karta hai ke naya data purane data ke sath kaise merge hoga - jaise messages list mein har naya message overwrite karne ki bajaye append ho jaye. Ye state management ko predictable banata hai.

Q12. Supervisor pattern kya hai multi-agent LangGraph mein?

Is pattern mein ek central 'supervisor' node hota hai jo decide karta hai ke current task ke liye kaunsa specialized agent (worker node) call hona chahiye. Worker apna kaam kar ke result wapis supervisor ko deta hai, jo phir next step decide karta hai - jab tak task complete na ho jaye.

3. LangSmith

LangSmith LangChain ka observability, debugging aur evaluation platform hai. Ye production mein chal rahe LLM applications ko monitor karne aur unki quality test karne ke liye use hota hai.

Q1. LangSmith kya hai aur kyun use karte hain?

LangSmith ek platform hai jo LLM applications ko trace, debug, test aur monitor karne ke liye banaya gaya hai. LLM apps mein bohot saare steps (prompts, tool calls, retrievals) hote hain jinhe normal logging se samajhna mushkil hota hai - LangSmith har step ka detailed trace deta hai taake developer easily dekh sake kahan issue aa raha hai.

Q2. Tracing kya hoti hai LangSmith mein?

Tracing ka matlab hai har LLM call, chain step, ya tool invocation ka pura record rakhna - input, output, latency, token usage, aur error agar koi ho. Ye ek tree structure ki tarah dikhta hai jisse pure execution flow ko visually trace kiya ja sakta hai.

Q3. LangSmith mein Evaluation kaise hoti hai?

Pehle ek dataset banaya jata hai jisme test inputs aur expected/reference outputs hote hain. Phir application ko un inputs pe run karke actual outputs generate karte hain. Evaluator (jo khud bhi ek LLM ho sakta hai - 'LLM-as-judge') outputs ko score karta hai - correctness, relevance, ya custom criteria ke basis pe.

Q4. LangSmith production monitoring mein kaise help karta hai?

Live traffic ke traces collect karta hai jisse error rate, average latency, cost (token usage) aur user feedback track kar sakte hain. Isse pata chalta hai application real users ke sath kaise perform kar raha hai, aur koi regression aaye to turant detect ho jata hai.

Q5. Dataset aur Evaluator mein kya farak hai?

Dataset test examples ka collection hota hai (inputs aur optionally expected outputs). Evaluator ek function ya LLM-based judge hota hai jo actual output ko check karta hai ke wo sahi hai ya nahi, aur ek score ya pass/fail result deta hai.

Q6. Debugging mein LangSmith ka sab se bara faida kya hai?

Complex multi-step chains ya agents mein agar final output galat aaye to pata lagana mushkil hota hai ke exactly kahan masla hua - retrieval mein, prompt mein, ya LLM reasoning mein. LangSmith ka step-by-step trace exactly wo node/step highlight kar deta hai jahan se cheezein galat hona shuru hui.

Q7. LangSmith mein Feedback collection kaise hoti hai?

End-users ke thumbs up/down reactions, ya custom numeric scores ko har trace ke sath attach kiya ja sakta hai. Is feedback se developers ko pata chalta hai ke production mein application real-world users ke liye kitni acchi perform kar rahi hai, aur weak areas identify ho jate hain.

Q8. LangSmith Playground kya hai?

Playground ek interactive UI hai jahan prompts, model parameters (temperature, max tokens, etc.) ko bina code likhe directly test aur tweak kiya ja sakta hai, aur turant result dekha ja sakta hai - rapid prompt engineering ke liye useful.

Q9. LangSmith se A/B testing kaise karte hain?

Ek hi dataset pe do alag prompts ya models run karke unke evaluation scores compare karte hain - is se data-driven decide kar sakte hain ke kaunsa version production ke liye behtar hai.

Q10. LangSmith open-source hai ya paid service?

LangSmith khud ek hosted/managed SaaS platform hai jisme free tier bhi available hai aur paid plans bhi, jabke core LangChain library (jo applications banane ke liye use hoti hai) fully open-source hai.

4. Vector Databases

Vector database ek specialized database hai jo high-dimensional vectors (embeddings) ko efficiently store aur search karne ke liye design ki gayi hai. RAG aur semantic search applications ki backbone yahi hoti hai.

Q1. Vector Database kya hota hai?

Vector database aisi database hai jo data (text, images, etc.) ko numerical vector (embedding) form mein store karti hai aur 'similarity search' perform karti hai - matlab kisi query vector ke sab se closest/relevant vectors dhundti hai, traditional keyword match ki bajaye semantic meaning ke basis pe.

Q2. Embeddings kya hote hain?

Embeddings text, image ya kisi bhi data ko ek fixed-length numerical vector mein convert karte hain jo us data ka semantic meaning capture karta hai. Jo cheezein meaning mein similar hoti hain unke vectors bhi numerical space mein ek dusre ke close hote hain.

Q3. Similarity Search kaise kaam karti hai?

Similarity search vectors ke beech distance ya similarity calculate karti hai - common metrics hain Cosine Similarity, Euclidean Distance, aur Dot Product. Jo vectors query vector ke sab se kareeb hote hain (ya jinka angle sab se kam hota hai), unhe most relevant result samjha jata hai.

Q4. Kuch popular Vector Databases ke naam batao.

Pinecone (managed cloud), Chroma (lightweight, open-source, local dev ke liye popular), Weaviate, Qdrant, Milvus, aur pgvector (PostgreSQL ka extension). FAISS Facebook ki ek library hai jo full database nahi balke efficient similarity search ke liye use hoti hai, often local/in-memory setups mein.

Q5. ANN (Approximate Nearest Neighbor) algorithms kya hote hain aur kyun zaroori hain?

Jab data millions vectors tak pohanch jata hai, to exact nearest neighbor search (brute-force comparison) bohot slow ho jati hai. ANN algorithms thori si accuracy qurban kar ke search ko bohot fast bana dete hain - is se large-scale applications real-time response de sakti hain.

Q6. HNSW kya hai?

HNSW (Hierarchical Navigable Small World) ek graph-based indexing algorithm hai jo ANN search ke liye widely use hota hai. Ye multiple layers ka graph banata hai jis se search bohot tezi se navigate ho kar nearest vectors tak pohanch jati hai, aur ye most vector databases ka default indexing method hai.

Q7. Vector database mein Indexing ka role kya hai?

Indexing data ko aise structure mein organize karti hai ke search karte waqt har vector ko individually check na karna pare. Achi indexing se search speed bohot improve hoti hai, especially jab dataset bara ho.

Q8. Metadata Filtering kya hota hai vector search mein?

Sirf vector similarity ke sath sath traditional filters bhi apply karna - jaise 'sirf un documents mein search karo jo last 6 months mein publish hue hon' ya 'category = finance' wale documents hi consider karo. Ye search ko zyada precise banata hai.

Q9. Chunking strategy RAG mein kyun important hai?

Documents ko bohot bara rakhne se relevant information dhundna mushkil hota hai, aur LLM ka context window bhi limited hota hai. Isliye documents ko chote, meaningful chunks mein todte hain (kabhi overlap ke sath, taake context na khoye), taake retrieval zyada accurate aur relevant ho.

Q10. FAISS aur Pinecone mein kya farak hai?

FAISS Meta (Facebook) ki open-source library hai jo local/self-hosted similarity search ke liye use hoti hai - isme khud infrastructure manage karna parta hai. Pinecone ek fully managed cloud service hai jo scaling, metadata filtering, aur real-time updates jaisi features readymade deta hai, lekin paid service hai.

Q11. Dense vs Sparse vectors mein kya farak hai?

Dense vectors (embeddings) har dimension mein meaningful values rakhte hain aur semantic/contextual meaning capture karte hain (jaise BERT, OpenAI embeddings). Sparse vectors (jaise TF-IDF, BM25) mostly zeros hote hain aur exact keyword matching pe based hote hain - in mein synonyms ya context samajh nahi aata, lekin exact terms (jaise product codes) dhundne mein behtar hote hain.

Q12. Hybrid Search kya hota hai?

Hybrid search dense (semantic) aur sparse (keyword-based) search dono ko combine karta hai, aur unke scores ko mix karta hai. Isse aisi queries bhi handle ho jati hain jin mein exact keyword match bhi zaroori ho aur semantic understanding bhi - jaise product names ya technical terms wali search.

Q13. Embedding ka dimension size kaise decide hota hai?

Ye embedding model decide karta hai - jaise OpenAI ka text-embedding-3-small 1536 dimensions deta hai, jabke kuch lightweight models 384 ya 768 dimensions use karte hain. Zyada dimensions zyada detail capture karte hain lekin storage aur compute cost barhati hain - isliye accuracy aur efficiency ke darmiyan tradeoff hota hai.

Q14. Re-ranking kya hota hai RAG pipeline mein?

Pehle vector search se top-k (jaise top-20) candidate documents fast retrieve karte hain. Phir ek aur, zyada accurate lekin slower model (jaise cross-encoder) in candidates ko dobara score karta hai aur sab se relevant ko top pe le aata hai. Ye final answer quality kafi improve kar deta hai.

5. LoRA (Low-Rank Adaptation)

LoRA (Low-Rank Adaptation) ek parameter-efficient fine-tuning (PEFT) technique hai jo large language models ko kam resources mein fine-tune karne deti hai.

Q1. LoRA kya hai?

LoRA (Low-Rank Adaptation) ek technique hai jisme large model ke original weights ko freeze (lock) kar diya jata hai, aur sirf chote, low-rank matrices add kiye jate hain jo training ke dauran update hote hain. Isse full model ko retrain karne ki zaroorat nahi parti, sirf chand additional parameters train hote hain.

Q2. LoRA mathematically kaise kaam karta hai?

Normal fine-tuning mein weight matrix W ka pura update (ΔW) train hota hai, jo bohot bara hota hai. LoRA is ΔW ko do chote matrices A aur B mein decompose kar deta hai (rank r ke sath), aise ke $\Delta W = B \times A$. Forward pass mein output $= W \times x + (B \times A) \times x$ hota hai. Sirf A aur B train hote hain, original W frozen rehti hai.

Q3. LoRA full fine-tuning se behtar kyun mana jata hai?

Kyunke isme train karne wale parameters ki tadaad bohot kam ho jati hai (kabhi 0.1% se bhi kam original model size ka), jisse memory aur compute requirement bohot kam ho jati hai. Original weights preserve rehte hain isliye multiple tasks ke liye alag alag chote LoRA adapters bana kar switch kiya ja sakta hai, bina poora model dobara save kiye.

Q4. Rank (r) parameter ka kya matlab hai LoRA mein?

Rank ye decide karta hai ke low-rank matrices A aur B kitni 'capacity' rakhti hain. Chota rank (jaise 4 ya 8) kam parameters add karta hai - training fast aur memory efficient hoti hai lekin model ki expressiveness kam hoti hai. Bara rank zyada parameters add karta hai - zyada capacity lekin overfitting aur compute cost ka risk bhi barhta hai.

Q5. LoRA typically model ke kis hisse pe apply hota hai?

Aam tor pe LoRA Transformer ke attention layers ki Query (W_q) aur Value (W_v) matrices pe apply hota hai, kyunke yahi sab se zyada task-specific behavior control karte hain. Kabhi kabhi sab linear layers (including feed-forward layers) pe bhi apply kiya jata hai better performance ke liye.

Q6. LoRA mein Alpha parameter ka kya role hai?

Alpha ek scaling factor hai jo LoRA ke update ($B \times A$) ki strength control karta hai final output mein. Practically scaling α/r ke form mein use hoti hai - is se training stability aur learning rate tuning aasan ho jati hai.

Q7. LoRA adapters ko 'merge' karna kya matlab hai?

Training ke baad, trained low-rank matrices ($B \times A$) ko original frozen weights (W) ke sath permanently add kar dete hain ($W_{\text{new}} = W + B \times A$). Is se inference ke waqt extra computation overhead nahi hota, model normal single-weight model ki tarah chalta hai.

Q8. LoRA catastrophic forgetting ko kaise reduce karta hai?

Chunki LoRA mein original pretrained weights bilkul change nahi hote (frozen rehte hain), model apni original general knowledge nahi bhoolta. Sirf naya, task-specific behavior chote additional

matrices ke through seekha jata hai, isliye full fine-tuning ke comparison mein forgetting ka risk kam hota hai.

Q9. PEFT kya hai aur LoRA iska part kaise hai?

PEFT (Parameter Efficient Fine-Tuning) ek broader category hai jis mein LoRA ke ilawa Prefix Tuning, Prompt Tuning, aur Adapter layers jaisi techniques bhi شامل hain. In sab ka common goal hai model ke kam se kam parameters update kar ke usay naye task pe adapt karna, taake training cheap aur fast ho.

Q10. LoRA training ke liye konse hyperparameters important hote hain?

Sab se important hain: rank (r) - capacity decide karta hai; alpha - update ki strength scale karta hai; target_modules - kis layers (W_q , W_v , etc.) pe LoRA apply karna hai; learning rate; aur dropout (overfitting kam karne ke liye).

Q11. Ek hi base model ke sath multiple LoRA adapters kaise use kar sakte hain?

Har specific task ya domain ke liye apna alag, chota LoRA adapter train kar lete hain (jaise ek customer support ke liye, ek summarization ke liye). Base model wahi rehta hai - runtime pe sirf zaroorat ke mutabiq adapter ko load/swap kar dete hain, jisse ek hi model multiple tasks efficiently handle kar leta hai.

6. QLoRA (Quantized LoRA)

QLoRA, LoRA ka aik advanced version hai jo quantization ke sath combine ho kar bohot bare models ko bhi limited GPU memory pe fine-tune karna mumkin banata hai.

Q1. QLoRA kya hai?

QLoRA (Quantized LoRA) ek technique hai jisme base model ke weights ko fine-tuning se pehle low precision (4-bit) mein quantize kar dete hain, taake memory usage kaafi kam ho jaye. Quantized frozen base model ke upar normal LoRA adapters add kar ke training hoti hai.

Q2. LoRA aur QLoRA mein basic farak kya hai?

LoRA mein base model normally full ya half precision (jaise 16-bit) mein hota hai. QLoRA mein base model ko 4-bit precision mein quantize kar dete hain, jisse memory footprint kaafi kam ho jata hai. LoRA adapters (jo train hote hain) dono cases mein higher precision mein rakhe jate hain taake training stable rahe.

Q3. NF4 (4-bit NormalFloat) kya hai?

NF4 ek special quantization data type hai jo specifically un weights ke liye optimize kiya gaya hai jo normal distribution follow karte hain (jaise neural network weights typically karte hain). Ye standard 4-bit integer quantization se behtar accuracy deta hai aur QLoRA ka core innovation hai.

Q4. Double Quantization kya hai?

Double quantization mein quantization process ke 'constants' (jo values quantize karne ke liye use hote hain) ko bhi dobara quantize kar dete hain. Isse per-model thore aur bytes ki memory bhi save hoti hai, jo bare models pe milakar significant ho jati hai.

Q5. Paged Optimizers QLoRA mein kyun use hote hain?

Training ke dauran kabhi kabhi memory usage achanak spike kar jata hai (jaise lambi sequences process karte waqt), jisse out-of-memory error aa sakta hai. Paged optimizers NVIDIA ki unified memory feature use karte hain jisse zaroorat parne pe data automatically GPU aur CPU memory ke darmiyan 'page' ho jata hai, crashes avoid karte hue.

Q6. QLoRA se bare models ko consumer GPU pe fine-tune karna kaise mumkin hota hai?

Quantization ki wajah se model ki memory requirement bohot kam ho jati hai - misaal ke tor pe ek 65 billion parameter model jo normally multiple high-end GPUs maangta, QLoRA ke through sirf ek single 48GB GPU pe fine-tune kiya ja sakta hai, bina performance mein zyada compromise ke.

Q7. Quantization se accuracy loss kyun hota hai aur QLoRA isay kaise minimize karta hai?

Kam bits (jaise 4-bit) mein numbers represent karne se precision kam ho jati hai, jisse model ki accuracy thori gir sakti hai. QLoRA is loss ko NF4 datatype (jo weights ki normal distribution ke liye optimized hai) aur double quantization jaisi techniques se minimize karta hai, taake performance full-precision ke kareeb rahe.

Q8. QLoRA training ke baad inference kaise karte hain?

Do options hote hain: ya to quantized base model ke sath hi LoRA adapters ko load kar ke inference karein (memory efficient, thori si speed kam ho sakti hai), ya LoRA weights ko higher precision mein merge kar ke ek normal model bana lein (thori zyada memory lekin faster inference).

7. Practical / Scenario-Based Questions

1-year experience wale candidates se aksar 'kya jante ho' se zyada 'real situation mein kya karoge' wale practical/scenario-based sawal puche jate hain. Ye section aise hi common scenario questions cover karta hai.

Q1. Agar aapka RAG system galat ya irrelevant answers de raha ho, to aap kaise debug karoge?

Sab se pehle LangSmith jaisi tool se trace dekhunga ke retrieval step mein sahi documents aa rahe hain ya nahi. Agar retrieval hi galat hai to chunk size/overlap aur embedding model check karunga. Agar retrieval sahi hai lekin LLM phir bhi galat jawab de raha hai, to prompt review karunga ke context properly inject ho raha hai ya nahi, aur LLM ko clearly instruct karunga ke sirf diye gaye context se answer kare.

Q2. Production mein LLM API costs ko aap kaise control karenge?

Caching implement karunga taake repeat queries pe dobara API call na lage, simple tasks ke liye chote/cheaper models use karunga, prompt ko concise rakh kar tokens kam karunga, aur per-user ya per-request token limits set karunga taake costs predictable rahein.

Q3. Agar chatbot hallucinate kar raha ho (galat/banayi hui information de raha ho) to kya approach lenge?

RAG implement karunga taake answers actual documents pe grounded hon, system prompt mein clearly likhunga ke 'sirf diye gaye context se jawab do, agar context mein answer na ho to bolo pata nahi', temperature kam rakhunga, aur jahan possible ho wahan source citation bhi include karunga.

Q4. LLM application ki latency kam karne ke liye konsi techniques use kar sakte hain?

Streaming response use karunga taake user ko word-by-word output dikhe, frequently puche jane wale queries ko cache karunga, simple steps ke liye chote/faster models use karunga, independent tool calls ko parallel mein run karunga, aur jahan mumkin ho async processing use karunga.

Q5. Agar aapko company ke internal documents pe ek chatbot banana ho, to apna approach explain karein.

Pehle documents collect aur clean karunga, phir unhe meaningful chunks mein todunga (with some overlap), har chunk ka embedding bana kar vector database mein store karunga. Phir ek retriever + LLM (RAG) pipeline banaunga, aur akhir mein ek evaluation dataset bana kar test karunga ke answers accurate hain ya nahi, phir feedback ke basis pe improve karunga.

Q6. Fine-tuning (LoRA/QLoRA) aur RAG mein se kab konsa use karna chahiye?

Agar knowledge frequently update hoti rehti hai (jaise company policies, latest data), ya factual grounding zaroori hai, to RAG zyada suitable hai kyunke isme model ko retrain nahi karna parta - bas documents update karte hain. Agar model ka tone, style, format, ya domain-specific language/behavior change karna ho, to fine-tuning (LoRA/QLoRA) zyada appropriate hota hai. Kabhi kabhi dono ko combine bhi karte hain.

Q7. Vector database mein search slow ho jaye to kya optimize karenge?

Indexing algorithm ke parameters (jaise HNSW ke ef_search, M) tune karunga, embedding dimensions kam karne ka option dekhunga agar accuracy pe zyada farak na pare, queries ko batch

mein process karunga, aur agar zaroorat ho to infrastructure scale (sharding/replication) karunga.

Q8. Prompt injection attack kya hota hai aur isay kaise prevent karte hain?

Prompt injection mein malicious user apne input mein aisi instructions daalta hai jo LLM ko uske original system instructions bhulane ya unke against jaane pe majboor kare. Prevention ke liye: user input ko clearly separate aur mark karein system instructions se, output ko validate karein, sensitive actions ke liye human approval rakhein, aur strong/specific system prompts use karein.

Q9. Agar LLM ka context window exceed ho jaye to kya karenge?

Text ko summarize ya important parts hi select karunga, sliding window approach use kar ke purani conversation ko summary mein convert karunga, ya phir bara context window wala model use karunga agar budget allow kare.

Q10. LLM application ki quality measure karne ke liye konse evaluation metrics use karte hain?

Retrieval ke liye precision/recall jaise metrics, generation quality ke liye LLM-as-judge scoring ya human evaluation, factual accuracy ke liye groundedness checks, aur saath hi latency aur cost bhi monitor karte hain - sirf accuracy hi kaafi nahi hoti production application ke liye.

8. Foundational AI/ML Concepts

LLM/AI roles mein, especially 1-year experience level pe, kuch foundational concepts bhi zaroor puche jate hain taake check ho sake ke basics clear hain ya nahi.

Q1. Transformer architecture basic taur pe kya hai?

Transformer ek neural network architecture hai jo 'Attention is All You Need' paper mein introduce hui thi. Ye self-attention mechanism use karta hai jisse model ek sentence ke har word ka relation dusre words ke sath simultaneously samajh sakta hai (sequential processing ki bajaye), jo isay LLMs jaise GPT, Llama, Claude ka foundation banata hai.

Q2. Attention mechanism kya hai aur ye kaise kaam karta hai?

Attention mechanism model ko ye decide karne deta hai ke ek word/token ko process karte waqt sentence ke kaunse dusre words zyada 'important' hain. Har token Query, Key, aur Value vectors banata hai, aur in ke beech similarity calculate kar ke relevant tokens ko zyada weight di jati hai.

Q3. Tokenization kya hoti hai?

Tokenization wo process hai jis se raw text ko chote units (tokens) mein toda jata hai - ye words, sub-words, ya characters ho sakte hain. LLMs text ko directly nahi samajhte, pehle isay tokens (numbers) mein convert karte hain phir process karte hain.

Q4. Encoder aur Decoder models mein kya farak hai?

Encoder models (jaise BERT) puri input sentence ko ek sath dekh kar uska representation banate hain - ye understanding tasks (classification, embeddings) ke liye behtar hote hain. Decoder models (jaise GPT) text ko left-to-right generate karte hain, ek time pe ek token, isliye ye text generation tasks ke liye use hote hain.

Q5. Supervised aur Unsupervised learning mein kya farak hai?

Supervised learning mein model ko labeled data (input + correct output) diya jata hai taake wo seekhe. Unsupervised learning mein model ko sirf input data diya jata hai aur wo khud patterns ya structure dhundta hai, bina kisi labels ke (jaise clustering).

Q6. Overfitting aur Underfitting kya hote hain?

Overfitting tab hoti hai jab model training data ko itna acche se yaad kar leta hai ke naye/unseen data pe achi performance nahi de pata. Underfitting tab hoti hai jab model itna simple ya kam trained hota hai ke training data pe bhi achi performance nahi de pata. Dono cases mein generalization (naye data pe performance) kharab hoti hai.

Q7. Temperature parameter LLM mein kya control karta hai?

Temperature LLM ke output ki randomness/creativity control karti hai. Kam temperature (jaise 0 ya 0.2) output ko zyada deterministic aur focused banati hai, jabke zyada temperature (jaise 0.8 ya 1.0) zyada diverse aur creative lekin kabhi kabhi inconsistent output deti hai.

Q8. Prompt Engineering kya hai?

Prompt Engineering wo skill hai jisme LLM se behtar, accurate aur consistent results lene ke liye prompts ko carefully design kiya jata hai - jaise clear instructions dena, examples (few-shot) include karna, ya step-by-step sochne ko kehna (chain-of-thought).

Q9. Context Window kya hota hai?

Context window ek LLM ki maximum capacity hai ke wo ek waqt mein kitne tokens (input + output combined) process kar sakta hai. Agar conversation ya document is limit se bara ho jaye, to purani information truncate/lost ho sakti hai.

Q10. API key ya credentials ko production mein secure kaise rakhte hain?

Environment variables ya secret management services (jaise AWS Secrets Manager, .env files jo git mein commit na hon) use karte hain, keys ko code mein hardcode nahi karte, aur access ko least-privilege principle ke according limit karte hain.

9. HR / General Interview Questions

Technical round ke sath sath, Lahore/Pakistan ki companies mein aksar ek HR ya general round bhi hota hai. Ye kuch common sawalat hain jo aapse puche ja sakte hain - inke jawab pehle se soch kar tayyar rakhna faida mand hoga.

Q1. Apne baare mein bataye (Tell me about yourself).

Short aur relevant rakho: apna educational background, jo skills/technologies (LangChain, vector databases, fine-tuning, etc.) seekhi hain unka zikar karo, agar koi project ya internship ki hai to uska short overview do, aur akhir mein bolo ke is role mein kyun interested ho.

Q2. Aapne is 1 saal mein kya seekha aur kis tarah ke projects pe kaam kiya hai?

Specific projects ka naam aur unka purpose batao (jaise 'maine ek RAG-based chatbot banaya jo company documents pe sawalat answer karta tha'), konsi technologies use ki (LangChain, vector DB, etc.), aur koi specific challenge jo solve kiya uska mention zaroor karo - numbers/results ke sath ho to behtar hai.

Q3. Aap is company mein kyun kaam karna chahte hain?

Company ke products/mission ke baare mein thori research kar ke jao, aur batao ke unka kaam aapki skills (LangChain/LLM applications) ke sath kaise align hota hai, aur aap unki team mein kya contribute kar sakte hain.

Q4. Aapki strengths aur weaknesses kya hain?

Ek genuine strength batao jo role se related ho (jaise 'naye tools jaldi seekh leta hoon'), aur ek aisi weakness batao jis pe aap actively kaam kar rahe ho (jaise 'pehle main documentation likhne mein time spend nahi karta tha, ab habit bana raha hoon').

Q5. Agar deadline tight ho aur kaam zyada ho, to aap kaise manage karenge?

Tasks ko priority ke hisaab se todna, kya zaroori hai pehle complete karna, agar zaroorat ho to team lead/manager ko proactively update dena, aur realistic timelines communicate karna - panic na karna, balke structured approach lena.

Q6. Aap team mein kaam karna pasand karte hain ya akele?

Honest jawab do, lekin flexibility dikhao - bolo ke complex problems pe team collaboration value karte ho (different perspectives milte hain), lekin focused deep-work tasks akele bhi efficiently kar lete ho.

Q7. Aapki salary expectation kya hai?

Pehle market research kar lo (Lahore mein 1-year experience AI/LLM roles ka range), aur ek range diya karo (exact number ki bajaye), aur ye bhi add karo ke aap flexible ho aur growth opportunities ko bhi value karte ho.

Q8. Kya aap remote, onsite ya hybrid work prefer karte hain?

Job posting ke according honest answer do - agar onsite role hai aur aap comfortable ho to clearly bolo, taake koi confusion na rahe.

Q9. Aap apne aap ko 2-3 saal baad kahan dekhte hain?

Realistic aur role-relevant answer do - jaise 'AI/LLM engineering mein deeper expertise build karna chahta hoon, aur eventually complex production systems lead karna chahta hoon' - growth-oriented lekin company ke sath align hona chahiye.

Q10. Koi sawal hai jo aap humse puchna chahte hain? (Do you have any questions for us?)

Hamesha kuch sawal tayyar rakho - jaise 'team is waqt konse projects pe kaam kar rahi hai?', 'role mein day-to-day kaam kaisa hota hai?', ya 'growth/learning opportunities kya hain?'. Koi sawal na puchna disinterested lagta hai.

Interview Tips

- Har answer ko apne alfaz mein, confidently, aur ek real example ke sath explain karne ki practice karo - sirf definition rata na lagao.
- Agar LangChain/LangGraph ka koi project pehle kiya hai to uska short walkthrough tayyar rakho (kya banaya, konse tools/models use kiye, kya challenge aaya, kya result mila).
- Vector database aur RAG ke concepts ko ek diagram (text-to-embedding-to-search-to-LLM) ki tarah zehan mein clear rakho, taake whiteboard pe bhi explain kar sako.
- LoRA aur QLoRA ka farak (precision aur memory ka) bohot common follow-up question hota hai - is point ko achi tarah clear rakho.
- 1-year experience ke role mein interviewer practical/scenario-based sawalat zyada puchta hai (jaise 'agar X masla ho to kya karoge') - sirf theory nahi, apna thought-process explain karna seekho.
- Apne kisi bhi project ka GitHub link ya demo tayyar rakho agar mauqa mile dikhane ka - Lahore ki companies isay positively dekhti hain.
- HR round ke common sawalat (apne baare mein, salary expectation, kyun ye company) ke jawab pehle se soch kar rakho, taake confidently bol sako.
- Agar koi sawal na aaye to honestly bolo aur apni reasoning/approach explain karne ki koshish karo - interviewer process dekhna chahta hai, sirf cram-shuda jawab nahi.

All the best for your interview kal!